

Semana 7 — Save/Load e consolidação do gameplay do Módulo 2

Semana 7

Save/Load e consolidação do gameplay do Módulo 2

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade II — Construir Sistemas (Semanas 4–7)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender serialização e recuperação de estado de jogo entre sessões como problema universal de qualquer engine
- Construir `SaveData` (Resource) + `FileAccess` como mecanismo de persistência real, e um `Checkpoint` que reutiliza o contrato `Interactable`
- Integrar todos os sistemas do Módulo 2 em um único fluxo jogável coerente, com Code Review e Playtest coletivo

Semana 7 — Save/Load e consolidação do gameplay do Módulo 2

ENCONTRO 1

SaveData, SaveComponent e Checkpoint

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 6 (`ItemData` + Enum, Checkpoint de progresso do Módulo 2) (15 min)
- Introdução: persistência entre cenas (`SaveManager`) versus persistência entre sessões (`SaveData` + `FileAccess`) (20 min)
- Demonstração: construção de `SaveData` , `SaveComponent` e gravação/leitura em `user://` (40 min)
- Laboratório: cada grupo implementa seu `SaveData` / `SaveComponent` salvando um dado real de progresso (40 min)
- Construção guiada da Scene `Checkpoint` , reutilizando o contrato `Interactable` (15 min)
- Feedback e fechamento (5 min)

Semana 7 — Save/Load e consolidação do gameplay do Módulo 2



Se o jogo é fechado e reaberto no dia seguinte, o progresso do jogador sobrevive?

Pense no que o `SaveManager` (Semana 4) já resolve — e no que ele não resolve.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Persistência Entre Sessões, Não Só Entre Cenas

- Toda engine com sessões de jogo separadas no tempo precisa transformar estado em memória em algo persistido em disco — e o processo inverso ao reabrir
- O `SaveManager` (Semana 4) resolve estado entre cenas; não resolve estado entre execuções do processo
- O Godot resolve isso com Resources serializáveis (`ResourceSaver` / `ResourceLoader`) ou `FileAccess` bruto, gravando em `user://`

SaveData : o Resource que Representa o Progresso

- Resource customizado, assim como `ItemData` (Semana 6) — mas grava e lê dinamicamente em `user://`, fora do projeto (`res://`)
- Campos exportados: itens coletados, último checkpoint alcançado
- Não é uma instância `.tres` criada manualmente — é instanciado e preenchido em tempo de execução pelo `SaveComponent`

Persistência em Disco no Godot × Unity

Godot

- `SaveData` (Resource) serializado nativamente via `ResourceSaver` / `ResourceLoader`
- `FileAccess` bruto disponível quando o arquivo precisa ser legível fora do motor

Unity

- `PlayerPrefs` para dados simples (chave-valor)
- Serialização própria em JSON/binário para dados estruturados — decisão inteiramente da equipe

Demonstração — SaveData e SaveComponent

O que será construído:

- Classe `SaveData` (Resource) com campos `itens_coletados` e `ultimo_checkpoint`
- `SaveComponent`, centralizando `salvar()` e `carregar()` via `ResourceSaver` / `ResourceLoader` em `user://`
- Teste de gravação e leitura, confirmando persistência entre execuções

Por quê: evita lógica de serialização duplicada entre Scenes — qualquer ponto do jogo que precise salvar conversa apenas com o `SaveComponent`.

Imagem sugerida

Objetivo: contrastar persistência entre cenas e persistência entre sessões.

Enquadramento: diagrama de duas colunas.

Elementos presentes: à esquerda, "SaveManager (Autoload)" com seta circular entre duas cenas do jogo em execução; à direita, "SaveData + FileAccess" com seta entre o jogo em execução e a pasta `user://`, e outra seta de volta ao reabrir o jogo.

Destaque visual: a pasta `user://` como o elemento que sobrevive ao fechamento do processo.

Legenda sugerida: "SaveManager mantém estado entre cenas; SaveData + FileAccess mantém estado entre sessões, gravando em disco."

Arquitetura — SaveComponent como Ponto Único de Gravação

- SaveComponent (Node): único responsável por ResourceSaver / ResourceLoader e pelo caminho user://save_data.tres
- Quem monta os dados a salvar (itens coletados, checkpoint) é responsabilidade de quem chama salvar(), não do Component
- Nenhuma Scene chama ResourceSaver / FileAccess diretamente — sempre através do SaveComponent

Checkpoint : o Contrato `Interactable` Aplicado à Persistência

- Scene que implementa o mesmo contrato `Interactable` já usado por `Door` e `Lever` desde a Semana 5
- Ao ser alcançado, aciona `SaveComponent.salvar()` — sem nenhuma lógica de interação própria
- Testa o desacoplamento ensinado na disciplina: um novo tipo de interativo não exige alteração no `InteractionComponent` do Player nem no contrato

Laboratório — SaveData , SaveComponent e Checkpoint

Cada grupo replica, no próprio projeto:

1. SaveData (Resource) em scripts/resources/save_data.gd , com itens_coletados e ultimo_checkpoint
2. SaveComponent em scripts/components/save_component.gd , com salvar() e carregar()
3. Teste de gravação/leitura, confirmando o arquivo em user://
4. Scene Checkpoint , reutilizando o contrato Interactable , acionando salvar()

Boas Práticas — Save/Load

- Manter o caminho do arquivo de save em uma única constante (`CAMINHO_SAVE`), nunca repetida como string literal
- `SaveData` enxuto: apenas dados que precisam sobreviver ao fechamento do jogo, nunca referências diretas a Nodes
- Um único `SaveComponent` ativo por vez, assim como um único `SaveManager`
- Dar feedback visual ou sonoro simples ao alcançar um `Checkpoint`

Fechamento — Encontro 1

- `SaveData` (Resource) e `SaveComponent` funcionais, salvando e recuperando progresso real entre sessões
- Ao menos uma Scene `Checkpoint`, implementando o contrato `Interactable` e acionando o `SaveComponent`
- `GameManager`, `SaveManager`, contrato `Interactable`, `Signals` e `ItemData/Enum` das Semanas 4 a 6 sem nenhuma alteração
- Próximo passo: integração completa do Módulo 2, no Encontro 2

Semana 7 — Save/Load e consolidação do gameplay do Módulo 2

ENCONTRO 2

Integração Final e Encerramento da Unidade II

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão integrada de GameManager, SaveManager, Interactable, Signals, Resources e save/load (15 min)
- Laboratório: integração final dos desafios do módulo em um único fluxo jogável (60 min)
- Preparação de cada grupo para apresentar e justificar sua integração (15 min)
- Code Review — cada grupo apresenta e justifica as escolhas de arquitetura (30 min)
- Playtest coletivo entre grupos e fechamento da Unidade II (15 min)



Cada sistema do módulo funciona sozinho — mas o jogador consegue percorrer o nível do início ao fim, sem que nenhum precise ser retrabalhado?

Pense em portas, alavancas, baús e o **Checkpoint** construídos em semanas separadas.

Integração: Um Único Fluxo Jogável

- Vertical Slice não é a soma isolada de sistemas — é a integração deles em uma experiência única
- Verifica se o desacoplamento ensinado desde a Semana 5 (contrato `Interactable`, `Signals`) permitiu montar tudo sem lógica duplicada
- Portas, alavancas, baús e `Checkpoint` conectados em um caminho único, do início ao fim

Arquitetura — O Fluxo Integrado do Módulo 2

- GameManager / SaveManager (Autoload) sustentam o estado global do fluxo
- Contrato Interactable + Signals conectam Player, portas, alavancas, baús e Checkpoint sem lógica duplicada
- ItemData /Enum definem os dados coletáveis; SaveData / SaveComponent persistem o progresso real
- Nenhum sistema exige alteração estrutural para se conectar aos demais

Imagem sugerida

Objetivo: ilustrar o fluxo jogável integrado que o grupo deve alcançar.

Enquadramento: mapa simplificado do nível de teste do grupo, visto de cima.

Elementos presentes: ícones de porta, alavanca, baú e checkpoint distribuídos no espaço, conectados por uma linha pontilhada representando o caminho jogável do início ao objetivo.

Destaque visual: a linha de fluxo contínua, mostrando que os sistemas não são ilhas isoladas.

Legenda sugerida: "Fluxo jogável integrado: cada sistema do Módulo 2 conectado ao caminho único do jogador."

Síntese Comparativa do Módulo 2 — Godot × Unity

Godot

- Autoload, contrato via duck typing/Orchestrator, Signals, Resource + Enum, `SaveData` /FileAccess

Unity

- Sem Autoload formal, Interfaces em C#, UnityEvent/C# Actions, `ScriptableObject` / `enum`, `PlayerPrefs` /JSON

Laboratório — Integração do Fluxo Completo

Cada grupo:

1. Posiciona/reposiciona `Door`, `Lever`, `Chest` e `Checkpoint`, formando um caminho único
2. Confirma que a alavanca controla a porta via Signal já conectado (Semana 5)
3. Confirma que o baú concede um item refletido no `GameManager` / `SaveManager`
4. Posiciona o `Checkpoint` em ponto lógico do caminho
5. Percorre o fluxo do início ao fim; fecha e reabre o jogo para confirmar persistência

Preparando a Justificativa de Arquitetura

- Para cada sistema do módulo, o grupo escreve uma frase: "por que fizemos dessa forma, e não de outra possível?"
- Distribuir a explicação entre integrantes — evitar que apenas uma pessoa saiba justificar as decisões técnicas
- Ensaiai mostrando o jogo sendo jogado do início ao fim, pausando em cada sistema

Boas Práticas — Code Review e Playtest

- Ensaiar com o jogo rodando de verdade, nunca descrevendo o código em abstrato
- Deixar o colega jogar livremente no Playtest, sem instruções verbais prévias
- Registrar objetivamente os pontos observados, mesmo sem tempo de corrigir agora
- Tratar feedback com o mesmo profissionalismo esperado em uma equipe real



Laboratório — Code Review e Playtest Coletivo

Apresente o fluxo jogável integrado ao professor, justificando as escolhas de arquitetura adotadas pelo grupo.

OBJETIVOS

Entrega: gameplay funcional consolidado do Módulo 2, avaliado por Code Review (Rubrica 4) e Playtest coletivo entre grupos.

Fechamento — Encontro 2

- Fluxo jogável único e coerente, sem sistemas isolados
- Progresso real persistido entre sessões, confirmado via **Checkpoint**
- Code Review (Rubrica 4) e Playtest coletivo realizados
- Unidade II — Construir Sistemas — encerrada

Resultado Esperado da Semana

- Vertical Slice com `GameManager` / `SaveManager`, contrato `Interactable`, `Signals`, `ItemData` /Enum e `SaveData` / `Checkpoint` integrados em um único fluxo
- Progresso real persistido entre sessões, não apenas entre cenas
- Turma relaciona `SaveData` /FileAccess ao equivalente da Unity (`PlayerPrefs` /JSON)
- Code Review e Playtest coletivo de encerramento do Módulo 2 concluídos

Checklist da Semana

- [] Classe `SaveData` (Resource) com `itens_coletados` e `ultimo_checkpoint`
- [] `SaveComponent` gravando e lendo corretamente em `user://`
- [] Progresso confirmado como persistente entre execuções (fechar e reabrir o jogo)
- [] `Checkpoint` implementando o contrato `Interactable`, sem lógica duplicada
- [] Portas, alavancas, baús e `Checkpoint` conectados em um único fluxo, do início ao fim
- [] Code Review (Rubrica 4) e Playtest coletivo realizados

Próximos Passos — Semana 8

O gameplay funcional consolidado nesta semana é a base direta da Semana 8, que inicia a Unidade III (Resolver Problemas) com o `HealthComponent` — reutilizando o mesmo padrão de Component já estabelecido pelo `SaveComponent` — e a fundamentação de AnimationTree/BlendSpace. A metodologia muda de Studio Based Learning para Challenge Based Learning: o professor apresenta problemas e os grupos propõem soluções com autonomia crescente.

Leitura recomendada: Godot Docs — Saving Games, Resources, File System; Unity Manual (consulta comparativa) — PlayerPrefs.